

React Lab: Event Handling, this, and Synthetic Events

Hands-on Lab Solution — "eventexamplesapp" (Counter, Say Welcome, Click, CurrencyConvertor)

Objectives

- Explain React events
- Explain about event handlers
- Define Synthetic event
- Identify React event naming convention

Lab Goals

- Implement the event handling concept in React applications
- Use the this keyword
- Use synthetic events

Prerequisites

- Node.js
- NPM
- Visual Studio Code

1. React Events

React lets you respond to user interactions — clicks, typing, form submission, mouse movement, and so on — by attaching event listeners directly to JSX elements, similar to plain HTML. Under the hood, React does not attach a listener to each individual DOM node; instead it registers a single listener at the root of the app and uses event delegation to route events to the right component, which is more efficient and lets React normalize event behavior across browsers.

The two main differences from plain HTML event handling are: (1) React events are named using camelCase (onClick, not onclick), and (2) you pass a function reference as the handler, not a string of executable code.

```
<!-- Plain HTML -->
<button onclick="increment()">Increment</button>

// React JSX
<button onClick={this.handleIncrement}>Increment</button>
```

2. Event Handlers

An event handler is simply a function that React calls when the associated event occurs. In class components, event handler methods do not automatically have this bound to the component instance, because JavaScript class methods are not bound by default. If an unbound method is used directly as a handler, this inside it will be undefined, and any access to this.state or this.setState will throw an error.

There are three common ways to make sure this is bound correctly inside an event handler:

- Bind the method in the constructor: `this.handleClick = this.handleClick.bind(this);`
- Declare the method as an arrow function class property: `handleClick = () => { ... }`
- Wrap the call in an inline arrow function in JSX: `onClick={() => this.handleClick()}`

This lab uses the constructor-binding approach so the role of the `this` keyword is explicit.

```
constructor(props) {
  super(props);
  this.state = { count: 0 };
  // Explicitly bind 'this' so handleIncrement can access this.state / this.setState
  this.handleIncrement = this.handleIncrement.bind(this);
}

handleIncrement() {
  this.setState({ count: this.state.count + 1 }); // 'this' now refers to the component
}
```

3. Synthetic Event

A `SyntheticEvent` is React's own cross-browser wrapper around the browser's native event object. Every event handler in React receives a `SyntheticEvent` instance (not the raw DOM event) as its argument. It exposes the same familiar interface as native events — `preventDefault()`, `stopPropagation()`, `target`, `type`, and so on — so handler code behaves consistently no matter which browser the app runs in. If the underlying native event is ever needed, it is still available through `event.nativeEvent`.

```
handleClick = (event) => {
  // 'event' here is a SyntheticEvent, not the raw browser event
  console.log(event.type);           // 'click'
  console.log(event.nativeEvent);    // the underlying native DOM event
  alert('I was clicked');
};
```

4. React Event Naming Convention

HTML	React (JSX)
onclick	onClick
onchange	onChange
onsubmit	onSubmit
onmouseover	onMouseOver
Attribute value is a string of code, e.g. onclick="doSomething()"	Attribute value is a reference to a function, e.g. onClick={doSomething}

Lab Steps

Step 1: Create the React application

```
npx create-react-app eventexamplesapp
cd eventexamplesapp
npm start
```

Open the eventexamplesapp folder in Visual Studio Code. Inside the src folder, create two new files: EventDemo.js and CurrencyConvertor.js.

Step 2: Increment / Decrement counter (Increment calls two methods)

The Increment button calls handleIncrement, which itself (a) updates the counter in state and (b) calls a second method, sayHello(), that logs a static greeting message. The Decrement button simply calls a second, separate handler.

src/EventDemo.js (Part 1: Counter)

```
import React, { Component } from 'react';

class EventDemo extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };

    // Bind 'this' inside the constructor
    this.handleIncrement = this.handleIncrement.bind(this);
    this.handleDecrement = this.handleDecrement.bind(this);
    this.sayHello = this.sayHello.bind(this);
    this.handleWelcome = this.handleWelcome.bind(this);
    this.handleClick = this.handleClick.bind(this);
  }

  // (a) increments the counter, (b) also calls sayHello()
  handleIncrement() {
    this.setState({ count: this.state.count + 1 });
    this.sayHello();
  }

  handleDecrement() {
    this.setState({ count: this.state.count - 1 });
  }

  // static message method invoked from handleIncrement
  sayHello() {
    console.log('Hello, this is a static message!');
  }
}
```

Step 3: “Say Welcome” button (passing an argument to the handler)

The Say Welcome button invokes handleWelcome, passing the string 'welcome' as an argument. Because the handler needs a custom argument, it is invoked from an inline arrow function in JSX rather than being passed directly as a bare reference.

```
// takes an argument and displays a welcome alert
handleWelcome(message) {
  alert('Say ' + message);
}
```

Step 4: Button using a Synthetic Event (“OnPress” / onClick)

The third button's onClick handler receives React's SyntheticEvent object as its parameter and simply displays “I was clicked”.

```
// receives a SyntheticEvent and displays a message
handleClick(event) {
  console.log('Synthetic event type:', event.type); // 'click'
  alert('I was clicked');
}
```

Step 5: Render the JSX for all three controls

```
render() {
  return (
    <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
      <h2>Counter: {this.state.count}</h2>
      <button onClick={this.handleIncrement}>Increment</button>
      <button onClick={this.handleDecrement}>Decrement</button>

      <h2>Welcome Button</h2>
      <button onClick={() => this.handleWelcome('welcome')}>
        Say Welcome
      </button>

      <h2>Synthetic Event Button</h2>
      <button onClick={this.handleClick}>Click Me</button>
    </div>
  );
}

export default EventDemo;
```

Step 6: Create the CurrencyConvertor component

CurrencyConvertor tracks the rupee amount typed into an input field in state, and converts it to euros when the Convert button is clicked. The button's onClick invokes handleSubmit, a synthetic event handler that receives the click's SyntheticEvent, calls event.preventDefault(), performs the conversion, and updates state with the result.

src/CurrencyConvertor.js

```
import React, { Component } from 'react';

class CurrencyConvertor extends Component {
  constructor(props) {
    super(props);
    this.state = { rupees: '', euro: null };

    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  // updates state as the user types, using the SyntheticEvent's target.value
  handleChange(event) {
```

```

    this.setState({ rupees: event.target.value });
  }

  // handles the Convert button's click (SyntheticEvent) and does the conversion
  handleSubmit(event) {
    event.preventDefault();
    const RATE = 0.011; // approximate INR -> EUR rate
    const euroValue = (Number(this.state.rupees) * RATE).toFixed(2);
    this.setState({ euro: euroValue });
  }

  render() {
    return (
      <div style={{ padding: '20px', fontFamily: 'Arial, sans-serif' }}>
        <h2>Currency Convertor (INR &rarr; EUR)</h2>
        <input
          type="number"
          value={this.state.rupees}
          onChange={this.handleChange}
          placeholder="Enter amount in Rupees"
        />
        <button onClick={this.handleSubmit}>Convert</button>

        {this.state.euro !== null && (
          <p>{this.state.rupees} INR = {this.state.euro} EUR</p>
        )}
      </div>
    );
  }
}

export default CurrencyConvertor;

```

Step 7: Update App.js

src/App.js

```

import React from 'react';
import EventDemo from './EventDemo';
import CurrencyConvertor from './CurrencyConvertor';

function App() {
  return (
    <div className="App">
      <h1>Event Examples App</h1>
      <EventDemo />
      <hr />
      <CurrencyConvertor />
    </div>
  );
}

export default App;

```

Step 8: Render with ReactDOM

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 9: Run the application

```
npm start
```

This opens the app at <http://localhost:3000>.

Expected Output

- Clicking Increment raises the counter by 1 and logs “Hello, this is a static message!” to the console.
- Clicking Decrement lowers the counter by 1.
- Clicking Say Welcome shows an alert reading “Say welcome”.
- Clicking Click Me shows an alert reading “I was clicked”.
- Typing an amount (e.g. 1000) into the Currency Convertor input and clicking Convert displays “1000 INR = 11.00 EUR”.

Summary

- React events are attached in JSX using camelCase names (onClick, onChange, onSubmit) and are passed a function reference, not a string.
- Event handlers are ordinary functions; in class components, this must be bound (via the constructor, arrow-function class properties, or inline arrows) so this.state and this.setState work inside them.
- Every React event handler receives a SyntheticEvent — React's cross-browser wrapper with the same interface (preventDefault(), target, type) as a native DOM event, accessible via event.nativeEvent if needed.
- Handlers can call other methods (as Increment calls sayHello) and can be invoked with custom arguments via an inline arrow function, as used for Say Welcome.
- The CurrencyConvertor component ties this together: onChange keeps state in sync with the input, and onClick on the Convert button calls handleSubmit, a synthetic event handler that performs the INR-to-EUR conversion.